QAOA testing- q on extracting parameters, inconsistent output states

https://github.com/tensorflow/quantum/issues/176

ROW 8

QAOA testing- q on extracting parameters, inconsistent output states #176

Closed

hblxuor opened on Mar 27, 2020 · edited by hblxuor — Edits

Hi, I'm trying out TFQ with the QAOA max-cut circuit that appears in arXiv:2003.02989v1. My set up is tensorflow_quantum 0.3.0, tensorflow 2.1.0 and cirq 0.8.0. I'm running on a CPU, on both a mac 10.13 and a linux box RHEL7.7. Both show the same results.

This ticket uses QAOA terminology, but the gist is tfq.layers.Sample is producing weird results for me, and model.weights appears to go in reverse order to match to sympy parameters.

TFQ works great on the QAOA execution proper- it minimizes the energy quickly. However, I run into trouble when I want to extract the final states- I want to check that the energy minimization corresponds with the maxcut solution. I'm doing this two different ways:

1. use tfq.layers.Sample after model fitting on the QAOA circuit to extract the final state, supplying the trained model weights as parameters.
2. build an identical circuit using cirq and again supply the trained model weights as parameters.

I did this for QAOA (p=1) on a trivial graph: two nodes, 1 edge. The solution state is (0,1) or (1,0), with optimal parameters gamma = -pi/2, beta = pi/8. What I found:

- tfq.layers.Sample produces completely random states. The average energy it predicts doesn't match the loss (= average energy for optimal value=0?).
- the cirq circuit works perfectly if I flip the weights. So if my parameter list is [gamma, beta], I have to assign weights as gamma = weight[1], beta = weight[0].

I tried flipping the weights for tf.Sample, but I still got random results.

I've been having trouble fully understanding the TFQ API, so this could be completely on me, but I don't understand the inconsistencies here, or why the model weights are flipped, if in fact they are. Thanks!
google_qaoa.py.zip

hblxuor on Mar 27, 2020 — Author

Here are some sample outputs I'm getting. The format is "[q1,q2] energy". The minimum energy is 0. The ordering is tf.Sample with ordered weights, tf.Sample with reversed weights, cirq prediction with reversed weights.

...
Epoch 1000/1000
1/1 [==============================] - 0s 2ms/sample - loss: 2.2680e-05

straight
WARNING:tensorflow:From /Users/gm102/anaconda3/lib/python3.7/site-packages/tensorflow_core/python/util/deprecation.py:507: calling count_nonzero (from tensorflow.python.ops.math_ops) with axis is deprecated and will be removed in a future version.
Instructions for updating:
reduction_indices is deprecated, use axis instead
[1 0] 0.0
[1 1] 1.0
[0 1] 0.0
[1 1] 1.0
[1 1] 1.0
[0 1] 0.0
[1 1] 1.0
[1 0] 0.0
[1 0] 0.0
[1 1] 1.0

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull r

Notifications
You're not receiving n
thread.

Participants

hblxuor on Mar 27, 2020 — Author

Here are some sample outputs I'm getting. The format is "[q1,q2] energy". The minimum energy is 0. The ordering is tf.Sample with ordered weights, tf.Sample with reversed weights, cirq prediction with reversed weights.

...
Epoch 1000/1000
1/1 [==============================] - 0s 2ms/sample - loss: 2.2680e-05

straight
WARNING:tensorflow:From /Users/gm102/anaconda3/lib/python3.7/site-packages/tensorflow_core/python/util/deprecation.py:507: calling count_nonzero (from tensorflow.python.ops.math_ops) with axis is deprecated and will be removed in a future version.
Instructions for updating:
reduction_indices is deprecated, use axis instead
[1 0] 0.0
[1 1] 1.0
[0 1] 0.0
[1 1] 1.0
[1 1] 1.0
[0 1] 0.0
[1 1] 1.0
[1 0] 0.0
[1 0] 0.0
[1 1] 1.0
reversed
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
[0 1] 0.0
[1 1] 1.0
[0 1] 0.0
[1 1] 1.0
[1 0] 0.0
[1 1] 1.0
[1 1] 1.0

[1. 0.] 0.0
[1. 0.] 0.0
[1. 0.] 0.0
[0. 1.] 0.0
[1. 0.] 0.0
[0. 1.] 0.0
[1. 0.] 0.0
[1. 0.] 0.0
[0. 1.] 0.0
[1. 0.] 0.0

MichaelBroughton on Mar 27, 2020 · edited by MichaelBroughton — Edits — Collaborator

Quickly glancing at your code It looks like your `model_circuit` and `myCircuit` are two very different circuits. Looking at your code I get model_circuit:

```
(0, 0): ──X──Z^(-0.159154943091895*gamma)──X──Z^(-0.159154943091895*gamma)──@──────────────────@──F
                                                                            │                  │
(0, 1): ────────────────────────────────────────────────────────────────────X──Rz(1.0*gamma)──X──────
```

Which might be missing an initial H gate wall when it goes through the sample layer, and then later on myCircuit:

```
(0, 0): ──H──ZZ──────Rx(1.75π)──M('m')──
          │                     │
(0, 1): ──H──ZZ^0.5──Rx(1.75π)──M───────
```

MichaelBroughton on Mar 27, 2020 · edited by MichaelBroughton — Edits — Collaborator

Quickly glancing at your code It looks like your `model_circuit` and `myCircuit` are two very different circuits. Looking at your code I get model_circuit:

```
(0, 0): ──X──Z^(-0.159154943091895*gamma)──X──Z^(-0.159154943091895*gamma)──@──────────────────@──
                                                                            │                  │
(0, 1): ────────────────────────────────────────────────────────────────────X──Rz(1.0*gamma)──X──
```

Which might be missing an initial H gate wall when it goes through the sample layer, and then later on myCircuit:

```
(0, 0): ──H──ZZ──────Rx(1.75π)──M('m')──
          │                     │
(0, 1): ──H──ZZ^0.5──Rx(1.75π)──M───────
```

Can you double check that this is the case on your end too ?

Another subtlety that you might be running into is that when using PQC, the order in which the symbol values are placed inside of the weight tensors is alphabetic with respect to symbol name. we initially assumed that the user wouldn't want to extract the symbols too often (otherwise they would use Expectation instead), which we have corrected here: #167 and will appear in a future release or the most recently nightly if you can't wait until then.

hblxuor on Mar 29, 2020 — Author

Can confirm different circuits. I took the first circuit directly from the QAOA example in the paper, so I'm confused by its strange form. There are spurious hadamards and CNOTs that I didn't specify that are clearly staggering the circuit. More trivially, the TFQ circuit's initial hadamards are inserted at the model Input layer while I manually inserted them in the cirq circuit; I'm assuming they're still being inserted correctly. Let me look more closely at this today.

Symbol order was definitely an issue- thank you for mentioning that. I'll just rename symbols and see if that helps in fixing things.

hblxuor on Mar 30, 2020 · edited by hblxuor — Edits — Author

TFQ is creating circuits that I don't fully understand. Here are some test circuits:

Using the attached file, which only uses identity gates on both qubits, I get:

```
QAOA TFQ circuit: (0, 0):
──X──Z^(-0.636619772367581*AAA)──X──Z^(-0.636619772367581*AAA)──X──Z^(-0.636619772367581*BBB)──X──Z^(-0
.636619772367581*BBB)──
```

and no output for qubit (0,1)
exptst.py.zip

When I perform X1 + X2 by changing the tst1 circuit to `tst1 += cirq.PauliString(cirq.X(qubit))`
I get
`QAOA TFQ circuit:
(0, 0):
──────H──────Rz(2.0*AAA)──────H──X──────Z^(-0.636619772367581*BBB)
──────X──────Z^(-0.636619772367581*BBB)──────

(0, 1):
──H──────Rz(2.0*AAA)──────H──
──`

When I make the tst1 circuit identities and change tst2 to
`st2 += cirq.PauliString(cirq.Z(qubit1)*cirq.Z(qubit2))` I get

```
QAOA TFQ circuit: (0, 0):
──X──Z^(-0.636619772367581*AAA)──X──Z^(-0.636619772367581*AAA)──@──────────────────@──X──Z^(-0.3183098861
83791*BBB)──X──Z^(-0.318309886183791*BBB)── │                  │ (0, 1):
```

TFQ is creating circuits that I don't fully understand. Here are some test circuits:

Using the attached file, which only uses identity gates on both qubits, I get:

```
QAOA TFQ circuit: (0, 0):
──X──Z^(-0.636619772367581*AAA)──X──Z^(-0.636619772367581*AAA)──X──Z^(-0.636619772367581*BBB)──X──Z^(-0
.636619772367581*BBB)──
```

and no output for qubit (0,1)
exptst.py.zip

When I perform X1 + X2 by changing the tst1 circuit to `tst1 += cirq.PauliString(cirq.X(qubit))`
I get
`QAOA TFQ circuit:
(0, 0):
──────H──────Rz(2.0AAA)──────H──X──────Z^(-0.636619772367581BBB)
──────X──────Z^(-0.636619772367581*BBB)──────

(0, 1):
──H──────Rz(2.0*AAA)──────H──────────────────────────────────────
──`

When I make the tst1 circuit identities and change tst2 to
`st2 += cirq.PauliString(cirq.Z(qubit1)*cirq.Z(qubit2))` I get

```
QAOA TFQ circuit: (0, 0):
──X──Z^(-0.636619772367581*AAA)──X──Z^(-0.636619772367581*AAA)──@──────────────────@──X──Z^(-0.3183098861
83791*BBB)──X──Z^(-0.318309886183791*BBB)── │                  │ (0, 1):
──────────────────────────────────────────────────────────────────X──Rz(2.0*BBB)──X──────────────────────────
──────────────────────────────────────
```

I think algebraically it comes out the same within a global phase, but I'm not sure. It's certainly not a direct translation.

Finally, when I fixed the symbols, the cirq circuit works perfectly with the trained parameters, but the output of the tfq circuit is still faulty. This is baffling to me, because the model is using the tfq circuit to correctly optimize the parameters. The cirq circuit is just to verify. The only thing that's not working for me is using tfq to verify that the output states are correct. Am i using tf.layers.Sample incorrectly? I'm at a loss.

hblxuor on Mar 30, 2020 · edited by hblxuor Edits Author ···

I'll attached a version of my qaoa code with the fixed symbols to verify. Again, everything is working perfectly except for sample. All I want to do is observe samples of states generated by a given parameter set, so if there's a better alternative or if I'm using Sample wrong, that's fine.

`QAOA TFQ circuit:
(0, 0):
──────@──────────────────────────────────────@──────X──────Z^(-0.159154943091895gamma)──────X──────Z^(-0.15915
4943091895gamma)──────H──────Rz(2.0zbeta)──────H──
| |
(0, 1):
──────X──────Rz(1.0gamma)──────X──
──H──
────Rz(2.0*zbeta)──────H──────

...
Epoch 1000/1000
1/1 [==============================] - 0s 2ms/sample - loss: 2.3752e-05
gamma, zbeta: [array([1.580253 , 1.1776009], dtype=float32)]
WARNING:tensorflow:From /Users/gm102/anaconda3/lib/python3.7/site-packages/tensorflow_core/python/util/deprecation.py:507: calling count_nonzero (from tensorflow.python.ops.math_ops) with axis is deprecated and will be removed in a future version.
Instructions for updating:
reduction_indices is deprecated, use axis instead
[1 0] 0.0
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
[1 0] 0.0

hblxuor on Mar 30, 2020 · edited by hblxuor

Edits ▾ Author ···

I'll attached a version of my qaoa code with the fixed symbols to verify. Again, everything is working perfectly except for sample. All I want to do is observe samples of states generated by a given parameter set, so if there's a better alternative or if I'm using Sample wrong, that's fine.

```
`QAOA TFQ circuit:
(0, 0):
———@———————————————————————————@———X———Z^(-0.159154943091895gamma)———X———Z^(-0.15915
4943091895gamma)———H———Rz(2.0zbeta)———H———————————————————————————————
| |
(0, 1):
———X———Rz(1.0gamma)———X———————————————————————————————————————————————
———————————————————————————————————————————————————————————————————H—
——Rz(2.0*zbeta)———H———

...
Epoch 1000/1000
1/1 [==============================] - 0s 2ms/sample - loss: 2.3752e-05
gamma, zbeta: [array([1.580253 , 1.1776009], dtype=float32)]
WARNING:tensorflow:From /Users/gm102/anaconda3/lib/python3.7/site-
packages/tensorflow_core/python/util/deprecation.py:507: calling count_nonzero (from tensorflow.python.ops.math_ops) with
axis is deprecated and will be removed in a future version.
Instructions for updating:
reduction_indices is deprecated, use axis instead
[1 0] 0.0
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
[1 0] 0.0
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
[1 1] 1.0
0:
[1]
1:
[0]
average energy

[now use cirq with tfq model-generated parameters]
gamma,zbeta = 1.580253005027771 1.1776008605957031
100
0.0
sample ket outputs
gamma,zbeta = 1.580253005027771 1.1776008605957031
(0, 0): ———H———ZZ——————————Rx(0.75π)———M('m')———
| |
(0, 1): ———H———ZZ^0.503———Rx(0.75π)———M——————————
10
[0. 1.] 0.0
[0. 1.] 0.0
[1. 0.] 0.0
[0. 1.] 0.0
[0. 1.] 0.0
[0. 1.] 0.0
[1. 0.] 0.0
[0. 1.] 0.0
[0. 1.] 0.0
[1. 0.] 0.0
0.0

`
```

fix_google_qaoa.py.zip

MichaelBroughton on Mar 30, 2020 · edited by MichaelBroughton — Edits — Collaborator

On L101 in the source file you linked you are not putting the hadamard wall before `model_circuit` like you were doing before. `sample_layer` is not connected to the previous `model` variable you created, nor does it acquire the hadamard wall from the `input_` tensor. I don't know what output you are shooting for here, but it looks like doing this at least gets the two output lists you have printing out to agree with one another:

```
sample_layer = tfq.layers.Sample()
outket = sample_layer(hadamard_circuit + model_circuit,
        symbol_names=qaoa_parameters,
        symbol_values=paramVals,
        repetitions=10)
```

Does this help with your issue at all ?

hblxuor on Mar 30, 2020 — Author

Yes! That (along with the symbol ordering) fixes it completely. As you said, I failed to realize the input circuit was not being sent to Sample. I don't fully understand why the circuit diagrams look the way they do, but I can live with that. I've tested this on larger graphs and everything is self-consistent. Thanks!

MichaelBroughton on Mar 30, 2020 — Collaborator

Great! Closing.

MichaelBroughton closed this as completed on Mar 30, 2020

jaeyoo added a commit that references this issue on Mar 30, 2023

`Merge pull request tensorflow#176 from MichaelBroughton/daggers ...` Verified `698e7e1`